Resume Database: Build Overview

1. What you are building

Two flows:

1. **Ingestion:** resume file in, structured searchable record out.
2. **Matching:** job description in, top 5 candidates with % scores out.

The hard part is not the web app. It is (a) parsing messy resumes reliably and (b) producing match scores recruiters trust. Spend most of your effort there.

2. Decisions to make before coding

- **Volume:** hundreds, thousands, or tens of thousands of resumes? Under about 50k, one Postgres database handles everything.
- **Users:** only Salam recruiters, or clients too? This changes your permission model.
- **Resume formats:** PDF and DOCX mostly? Scanned or image PDFs need OCR.
- **Consent and privacy:** do candidates know their resumes are stored and searched by software? Resumes contain sensitive personal data.
- **What "match" means to recruiters:** ask 2 or 3 recruiters to rank real candidates for a real job. You will use that as your test set.

3. Architecture

Browser (Next.js)
  |
API server (FastAPI)
  |-- File storage (S3 / Supabase Storage): original resume files
  |-- Postgres + pgvector: structured data, text, embeddings
  |-- Background worker: parsing, extraction, embedding
  |-- LLM API (Claude): extraction, JD analysis, reranking, explanations

4. Recommended tech stack

MVP (fastest path):

Layer	Choice	Why
Frontend	Next.js + TypeScript + Tailwind + shadcn/ui	Fast to build, good table and upload components
Backend	Python FastAPI	Best ecosystem for parsing and AI work

Database	Postgres with pgvector	Relational data, keyword search, and vector search in one system
Auth, storage, DB hosting	Supabase	Gives you Postgres, pgvector, auth, file storage, and row-level security in one place
Background jobs	Celery + Redis, or a managed option like Inngest	Parsing should not block uploads
LLM	Claude API	Structured extraction, JD parsing, reranking, explanations
Embeddings	A hosted embedding model (e.g., Voyage, OpenAI, or Cohere)	Anthropic does not offer its own embeddings, so pick one provider
Hosting	Vercel (frontend) + Render, Fly.io, or Railway (API)	Low ops overhead

Parsing libraries:

- PDF text: PyMuPDF or pdfplumber
- DOCX: python-docx
- Scanned PDFs and images: AWS Textract, Google Document AI, or Tesseract (cheaper, less accurate)

Alternatives: a Microsoft shop could use Azure (Blob Storage, Azure AI Search, Azure OpenAI, Entra ID). If you already know Dataverse and Power Platform, that works for an internal tool, but custom ranking and vector search are harder there. Elasticsearch or OpenSearch is only worth it at large scale.

5. Data model

- **candidates:** id, name, email, phone, location, source, consent flag, created_at
- **resume_files:** id, candidate_id, storage_path, file_type, uploaded_by, version, uploaded_at
- **resume_text:** raw extracted text, parse quality score
- **candidate_profiles (structured):** title(s), total years of experience, skills (array), education, certifications, work history (JSON), summary
- **embeddings:** candidate_id, chunk type (summary, experience entry, skills), vector
- **jobs:** id, title, raw JD text, parsed requirements (JSON), created_by
- **match_runs:** job_id, candidate_id, score, score breakdown (JSON), explanation, rank, timestamp
- **audit_log:** who searched, viewed, or downloaded what, and when

- **users and roles:** admin, recruiter

Keep a normalized skills table with synonyms (JS, JavaScript, Node.js) so “React” matches “React.js”.

6. Ingestion pipeline

1. **Upload:** drag and drop, bulk upload (zip or multi-file), and optionally email intake later.
2. **Store** the original file and create a job in the queue.
3. **Extract text.** If the extracted text is nearly empty, route to OCR.
4. **Structured extraction with the LLM.** Send the text and ask for JSON matching your schema (name, contact, skills, titles, dates, education). Validate the JSON with Pydantic and retry on failure.
5. **Normalize:** standardize skills, compute total years from date ranges, standardize job titles.
6. **Embed** the summary, each job entry, and the skills block. Multiple vectors per resume work better than one.
7. **Deduplicate:** match on email, phone, or name plus similarity, then ask a human to confirm merges.
8. **Flag low-quality parses** for manual review instead of silently indexing garbage.

7. The matching engine (the core)

Use a hybrid pipeline, not just vector similarity.

Step 1: Analyze the job description. Use the LLM to extract structured requirements: must-have skills, nice-to-have skills, minimum years, titles, education, location or remote status, certifications.

Step 2: Retrieve candidates (fast, about 50). Combine:

- Vector search on the JD embedding
- Keyword search using Postgres full-text search
- Hard filters (location, work authorization, availability) where recruiters specify them

Step 3: Score each candidate with a weighted rubric. Example starting weights:

Component	Weight
Must-have skills coverage	35%
Semantic similarity (normalized)	20%
Relevant experience and seniority fit	20%

Title and industry relevance	15%
Nice-to-haves, education, certs	10%

Step 4: LLM rerank the top 10 to 15. Give the model the structured requirements and each candidate's profile. Ask it to score against the rubric, cite evidence from the resume, and list gaps. Then blend the rubric score with the LLM score.

Step 5: Return the top 5 with the % score, matched skills, missing skills, a 2 to 3 sentence explanation, and a link to the original resume.

Important caveats about the % score:

- Raw cosine similarity is not a percentage. Normalize it, or rely on the rubric for the displayed number.
- The score is relative to your candidate pool and your rubric, not an objective truth. Calibrate it against recruiter rankings and label it accordingly.
- Show the breakdown. Recruiters trust a score they can inspect.
- Have a "no good match" state. If the best candidate scores low, say so instead of forcing 5 results.

8. Frontend pages

- Login
- Upload (single, bulk, progress and parse status)
- Candidate library (search, filter, sort, table view)
- Candidate detail (parsed data side by side with original PDF, edit and correct fields)
- Match page (paste or upload JD, run match, see top 5 cards with scores and explanations)
- Saved searches and past match runs
- Admin (users, audit log, re-parse failed resumes)

Useful features: export shortlist to PDF or CSV, thumbs up/down feedback on each result (this becomes your tuning data), and the ability to adjust requirement weights before re-running.

9. Security, privacy, and compliance

- Encrypt in transit and at rest, use private storage buckets, and use signed URLs that expire.
- Role-based access, SSO if possible, and an audit log of views and downloads.
- Retention policy and a deletion workflow (candidates may request removal under GDPR or CCPA).
- Check your LLM and embedding vendors' data-retention terms. Use zero-retention or enterprise settings where available.

- **Bias:** exclude name, gender, age, photos, and similar fields from scoring. Keep a human in the loop. Some jurisdictions regulate automated hiring tools (for example, NYC Local Law 144 requires bias audits). Have Salam's legal counsel review before launch. This is not legal advice.

10. Evaluation (do not skip)

- Build a test set: 5 to 10 real (anonymized) job descriptions, each with recruiter-ranked candidates.
- Measure whether your top 5 overlaps with the recruiters' top 5 (precision at 5, NDCG).
- Measure parsing accuracy on about 50 resumes: check name, skills, dates, and titles by hand.
- Re-run these tests every time you change weights, prompts, or models.

11. Build roadmap

Phase 1 (week 1 to 2): Foundation. Auth, upload, file storage, text extraction, database schema.

Phase 2 (week 3 to 4): Intelligence. LLM extraction, normalization, embeddings, candidate library UI.

Phase 3 (week 5 to 6): Matching. JD parsing, hybrid retrieval, rubric scoring, LLM rerank, top 5 results page.

Phase 4 (week 7 to 8): Trust and polish. Score breakdowns, feedback buttons, audit log, dedupe, evaluation harness, tuning weights.

Phase 5 (later): Email intake, ATS integration, client-facing portal, candidate re-engagement, analytics.

12. Common pitfalls

- Trusting PDF text extraction. Multi-column resumes and tables break it, so always show the original beside the parsed data.
- Embedding the entire resume as one vector. Chunk by section.
- Showing a precise-looking score (87.3%) that is not calibrated. Round it and explain it.
- Skipping the feedback loop. Recruiter feedback is your best tuning data.
- Over-building. Start with Postgres and one LLM provider before adding more infrastructure.

13. Rough cost expectations

At a few thousand resumes, hosting is roughly $50 to $150 a month. LLM and embedding costs are typically cents per resume to ingest and cents per match run. OCR for scanned resumes is the main variable cost.

Where to start

Build the thin vertical slice first: upload 20 resumes, extract text, run LLM extraction into JSON, embed, paste one JD, and return 5 ranked results with a basic explanation. Once that works end to end, improve each piece using your test set.

If you tell me your expected resume volume, whether this is internal-only, and your preferred cloud (or whether you want the simplest setup), I can narrow the stack and write the database schema and the extraction and matching prompts.